# WASP Software Engineering Assignment


- **Author:** Janek Stoppkotte
- **Research area:** Language-based security
- **Specific focus:** Information-flow control


## 1. Introduction

I work in language-based security. Language-based security is a field in information security, that encompasses the relation between languages and security. One example and also the part that I am working on is Information-flow Control (IFC). The idea lies in labelling variables, i.e. inputs and outputs of a program with labels. The labels usually are on a lattice, i.e. have a partial ordering with a bottom and usually a top element. The control in information flow control comes from the policies that you can enforce on the labels. So in a basic setting variable might be labelled **secret** or **public**. When your passwords are labelled **secret** and the outputs that send anything to the internet are labelled **public**, then one can enforce that no **secret** variable should influence **public** variables, thereby preventing any passwords from being uploaded to the internet. An example of a program that violates this property would be:

```
public := false
if secret then public := true
```

This program leaks the secret into the public variable. IFC can be enforced statically, either by an external tool that analyses the source code or in some cases even during type-checking. But languages, such as JavaScript make this difficult. So alternatively labels can be assigned during runtime and the IFC is enforced dynamically. This changes what policies can be enforced. The strong guarantee, that secrets cannot leak under any circumstances is usually not enforceable so we usually ignore some or all termination channels. The property we look to enforce is TINI, termination-insensitive non-interference.

In the simplest case, this leads us to simply block any assignments that happen when the control flow depends on a secret value. So if you consider the if in our example, there we branch on ‘secret‘ which is not okay. But this blocks many interesting programs that do not really leak the secret. Say, public is never used to influence an output, then we really do not need to stop this branching. A more permissive strategy is permissive upgrade. This allows us to upgrade the label from **public** to **partially leaked**. It stops the program if the **partially leaked** variable then influences an output [Austin and Flanagan, 2010].

Our observation is that this is explainable with assigning not only the variables a label, but to assign each label another label again. The labels on labels approach has been explored before, but we realized that this discovery is not explored in the literature. We suspect that this can be used to explain a few generalizations and perhaps make the approach more general even. There might also be more application of labels-on-labels other than permissive upgrade.

The work is entirely theoretical but can be used as groundwork in practice. For classical dynamic IFC, there is an implementation called JSFlow that enforces IFC on JavaScript [Hedin et al., 2016].

---

## 2. Ideas that matter for my research

Since I am specifically doing theoretical computer science, agentic software engineering is very far away from my topic. Sure, I can use LLMs and do a bridge between agentic and security by employing security methods to defend against sloppy security, but still it was hard to find ideas that are relevant to my research. The least sketchy link is maybe property-based testing.

### Connection to Agentic Software Engineering

There are a few connections between my research area and agentic software engineering. The relation that is closest to home is the work of my supervisor. He explores the LLMbda Calculus, a simple language that includes lambda calculus and LLM primitives plus a rigorous notion of IFC [Garby et al., 2026]. The idea is to employ the dual-LLM pattern, a software pattern described by Simon Willison, to ensure that prompt injection cannot do too much damage [Willison, 2023]. Information-flow control is used to keep untrusted data from influencing the privileged LLM while also doing classical IFC to ensure secrets are not leaked.

More generally, AI slop is a good reason to employ methods that systematically enforce security properties on code. Just imagine an AI engineer that sloppifies entire codebases and creates pull requests without considering security. Here one can use rigorous methods such as proper IFC to make sure that the model will eventually return code that satisfies those properties. So eventually those methods can be built into code harnesses that trigger the agentic loop until the code is secure.

A big second thing that is relevant to my research is the social aspect. I will explain this further below, but in short, using LLMs for everything does make you lonely.

### Property-based testing

In one of the first lectures we talked about traditional software testing and quality assurance in general. The example was property-based testing, an idea that started as John Hughes' research here at Chalmers. It combines parameterized properties with random test-data generation [Feldt, Lecture 2, slides 23-24]. QuickCheck was introduced by Koen Claessen and John Hughes [Claessen and Hughes, 2000]. It does relate to my research insofar as it is also software analysis. One can for example formulate the TINI property that I mentioned earlier, as a property in the QuickCheck library. This does not prove TINI, but many passing generated examples can still be useful evidence that the code is fine.

### Social aspect

I suspect this to be a big influence on the researching itself, less on my research itself. Robert's Behavioral SE slides say that technologies and processes work through their effects on engineers and communication [Feldt, Lecture 3, slide 18]. In the course discussion, as I remember it, we talked about junior developers no longer interacting with their seniors because they can ask their chatbot for help. Then over time, the only interaction you have with humans might be during lunch. This is very depressing and might make the job very undesirable. Realistically the same will be true for research. When I have questions about a paper, instead of asking the authors or colleagues I can ask Codex that has read the paper and perhaps all its sources. So in most cases I will probably quickly get an answer that usually is more correct than whatever my colleagues would say. So, why even bother asking them?

For now, the models do hallucinate quite a bit, so you can run the risk of one hallucination in the beginning, poisoning the entire session. Then because you trust the model more than your colleagues, maybe you start believing the hallucination. This is especially dangerous if the conversation has been going on for a long time.

I suspect that during my research I will need to find a middle ground. I should use Codex only when it accelerates my work without a big risk of rabbit holing me. Also, I should try to sync with colleagues, even if it just is to get a third opinion on something.

---

# 3. Two guest-lecture principles

## Automation and employment

One thing that I remember the speaker mentioned was the worry about agents replacing every job in software engineering. I also remember that he switched from software to electrical engineering because hardware is a more difficult terrain than working on software at Saab, where apparently Claude is required everywhere. The written slides do not contain this story, so this is my recollection of the lecture. They do say that AI can replace some informal coordination and make hidden gaps visible [Lenberg, 2026, slides 7-8]. In research agents are used to automate whatever aspect the authors find annoying. This can be literature research, i.e. the agent summarizes all related works. This can be making slides, posters or infographics. This can be writing the paper itself. This can be brainstorming the idea. This can also be a combination of some of them. Perhaps it also does the entire research. The big question is therefore, whether my research will be at all required in the coming years, or whether we got automated away. It could also be that we will not work in any intellectual job anymore, because the chatbots can do that for cheaper.

## Creativity

In the discussion, as I remember it, we also touched on creativity. Maybe LLMs cannot be creative and will always give the same ten ideas if asked. Then a human might still be required to give the right inputs, but implementation details are not relevant anymore. This connects to what my Unit discussed recently. It could be that I do not need to write proofs anymore, but rather have to present the idea and the creative aspect. The proof would still be expected because that part can be automated and serves as an artifact to validate your idea. This is the future that sounds more promising to me, so let's hope that this happens.

---

# 4. Data scientists, software engineers, and AI engineers

A data scientist focusses on different things than a software engineer. Lots of their focus is spent on selecting the data, training models and experimenting. Software engineers focus on requirements, architecture, testing, deployment and maintenance [Kästner, 2025, Chapter 1].

This distinction is soft, and in the real world I do expect to see people that do both. Data scientists can also use more than just notebooks, and some of them also work test-driven. How much software engineers focus on maintenance is also questionable. This view has no direct relevance to my research area, because my work is theoretical. It can matter if my work is implemented in a production system, because the security property then has to hold for the whole system and not only one model or component [Kästner, 2025, Chapter 2]. Language-based security could maybe play a role in software engineering, because my research extends its theoretical foundation. Software engineers might use that foundation in production systems.

I think software engineers and data scientists will remain different roles, since data scientists usually solve mathematical problems, whereas software engineers write software. So it is a different skillset that makes a good data scientist or a good software engineer. They still need enough shared knowledge to work together, but that does not make them the same role.

AI engineering as a term should maybe capture the people that are somewhere between data scientists and software engineers. But it seems to me that this is a marketing term without real value. Foundation models, coding assistants, RAG systems, prompt pipelines and agentic workflows do not change my answer because the underlying tasks remain different. They are tools that should make some processes faster since both groups can now ask a chatbot to do part of the work. Agentic software influences all of those areas similarly to other intellectual work, its disruption is not limited to these two roles. Talking about software engineering in the context of agentic software, may be more interesting but data scientists do not really play a role here.

# 5. Paper analysis

## Do Users Write More Insecure Code with AI Assistants?

Neil Perry et al. discuss whether AI coding-assistant usage has an effect on code security [Perry et al., 2023]. For this, they conducted a study on 47 participants where 33 had access to Codex whereas 14 wrote code without the assistant. The participants attempted five security tasks covering cryptography, path traversal, SQL injection and C memory management.

The authors come to the conclusion that participants with access to Codex produced less secure solutions in four of five tasks. However, they were still more likely to believe their solution to be secure. Participants that had a more interactive style where they re-prompted and questioned the model made fewer mistakes than the people that fully vibe-coded everything. They also discuss how reusing output from previous prompts could amplify mistakes. The study only had 47 participants, mostly students, doing artificial tasks with one version of Codex. So it is not clear that exactly the same result applies today. The paper is from 2023 instead of the preferred 2024-2026 range, but I chose it because it directly addresses security and is much closer to my research.

In software engineering, one should maybe retain an element of meaningful human control. If junior engineers generate security-relevant code, then a senior engineer has to verify it. Hence the model needs to be limited to writing compact code in order to not overwhelm the seniors.

For my research this is at most tangentially relevant. We work on methods that ensure code satisfies certain security properties by construction. If the agentic loop can use the static and dynamic analysis oracles, then the produced artifacts will become much better. So my research might matter to agentic development a lot.

An interesting but fictional AI-enabled project could be a smart home assistant. Imagine you have a smart home with a lot of security sensitive devices. This includes IP cameras, cleaning robots, voice assistants and more. You do not want that video or audio feeds can reach the outside world, via the internet for example. But if you use a voice assistant then your methods of nicely interacting with the smart home become much more interesting if the text input is processed with an LLM. This agent could however be vulnerable to prompt injection. So what you could do is enforce information flow control. All information that is labeled sensitive should then only be usable to influence local devices. Whenever the LLM gets sensitive information in its context, it cannot start any public actions. This could be achieved by using the project of my supervisor, the LLMbda Calculus [Garby et al., 2026]. My research would extend the labelling in that paper to permit more programs which do not actually harm the IFC policies that we want to enforce.

I cannot tweak my own research to better fit agentic software, other than to make it more applied. This may be follow up work, but is not relevant for the topic just now. It is also perfectly fine not to use agentic software, LLMs or any other ”AI” in one's research just because agentic software is disrupting the market.

### A Meta-Summary of Challenges in Building Products with ML Components - Collecting Experiences from 4758+ Practitioners

This paper combines 50 relevant papers into a meta-summary [Nahar et al., 2023]. In total, the studies interacted with more than 4,758 practitioners. The authors used systematic literature review guidelines and grouped more than 500 mentions of challenges. This leads to the central conclusion that machine learning does not only add a software component to a software project, but makes many steps of the classical software engineering pipeline more uncertain. Requirements, testing, architecture, maintenance and planning all have to handle the ML component's uncertainty.

The authors discuss that many teams focus only on model accuracy and thereby neglect concerns such as integration, monitoring, privacy and even security. Another problem is quality assurance. A probabilistic component makes testing harder. The same goes for versioning. Changing the training data or preprocessing steps can have unintended changes in system behavior. This can happen even when the rest of the system remains unchanged.

ML projects need cooperation between data scientists, software engineers and managers. Since they use different terminology and priorities, they might have problems about how to approach projects. A lot of energy may be spent trying to sync and divide up work. Another problem is that security, privacy and fairness may be neglected because organizations want faster project development and focus more on tangible things. They prefer more features over stuff that is hard to see.

A note from me: This is not really limited to products with ML components, but is a general software engineering problem. However, especially fairness is a bigger problem, as bias in the training data will be reflected in the product.

Again the connection to my research is not really there and at best, my methods can be used to make an ML product guarantee certain security properties. So I can help with the quality assurance aspect that was discussed in the paper.

A larger AI project here could be an organization-wide chatbot that has access to internal APIs, documents and chats. Then with an IFC labelling for every item in the organization we could formulate a policy that the chatbots harness would guarantee by construction. The bot could generate programs and execute them, while guaranteeing that the policies hold.

Adapting my research like before is probably a bad idea, but I could focus on system-wide constraints and practically analyze how to label everything in an organization such that a chatbot can interact with it without losing too much utility. One limitation is that the 50 papers had different scopes and reported their results differently. So we cannot tell how common or severe a challenge is from the number of mentions.

## 6. Evidence, originality, and GenAI-use transparency

### Source assessment

The two papers in Section 5 are peer-reviewed conference papers, so I trust them most for the empirical claims they actually studied. They still have the limitations that I mentioned. The CMU book chapters and the lecture slides are course sources and I use them for the course concepts and questions. The LLMbda Calculus is a preprint and Simon Willison's description of the dual-LLM pattern is a blog post. I only use them to describe the ongoing project and where the pattern came from. Claims from spoken discussions are marked as my recollection.

### GenAI disclosure

For this essay I used Codex to help me search for CAIN papers that were at least in the slightest related to my research. I then read the sources independently and prepared an outline of the summary.

Based on the outline, I wrote my own summaries and analyses. Codex then checked that I met the criteria, checked source details and checked spelling and grammar. Where a sentence needed clarification, Codex helped rewrite it, but the analysis is still all mine.

## References


- Austin, Thomas H., and Cormac Flanagan. 2010. ”Permissive Dynamic Information Flow Analysis.” *Proceedings of PLAS 2010*, 3-12. https://doi.org/10.1145/1814217.1814220. Peer-reviewed conference paper.
- Claessen, Koen, and John Hughes. 2000. ”QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs.” *Proceedings of ICFP 2000*, 268-279. https://doi.org/10.1145/351240.351266. Peer-reviewed conference paper.
- Feldt, Robert. 2026. *Quality Assurance in Software Engineering and Machine Learning*. WASP Software Engineering course, Lecture 2, especially slides 23-24. Course slides.
- Feldt, Robert. 2026. *Behavioral Software Engineering*. WASP Software Engineering course, Lecture 3, especially slide 18. Course slides.
- Garby, Zac, Andrew D. Gordon, and David Sands. 2026. ”The LLMbda Calculus: AI Agents, Conversations, and Information Flow.” arXiv:2602.20064. https://arxiv.org/abs/2602.20064. Preprint.
- Hedin, Daniel, Luciano Bello, and Andrei Sabelfeld. 2016. ”Information-Flow Security for JavaScript and Its APIs.” *Journal of Computer Security* 24. https://doi.org/10.3233/JCS-160544. Peer-reviewed journal paper.
- Kästner, Christian. 2025. *Machine Learning in Production: From Models to Products*, Chapters 1-2. https://mlip-cmu.github.io/book/. Online textbook.
- Lenberg, Per. 2026. *Importance of Human Aspects in Software Engineering, or Importance of Behavioral Software Engineering*. WASP guest lecture, especially slides 7-8. Guest-lecture slides.
- Nahar, Nadia, Haoran Zhang, Grace Lewis, Shurui Zhou, and Christian Kästner. 2023. ”A Meta-Summary of Challenges in Building Products with ML Components - Collecting Experiences from 4758+ Practitioners.” *Proceedings of CAIN 2023*, 171-183. https://doi.org/10.1109/CAIN58948.2023.00034. Peer-reviewed long conference paper.
- Perry, Neil, Megha Srivastava, Deepak Kumar, and Dan Boneh. 2023. ”Do Users Write More Insecure Code with AI Assistants?” *Proceedings of CCS 2023*, 2785-2799. https://doi.org/10.1145/3576915.3623157. Peer-reviewed conference paper.
- Willison, Simon. 2023. ”The Dual LLM Pattern for Building AI Assistants That Can Resist Prompt Injection.” https://simonwillison.net/2023/Apr/25/dual-llm-pattern/. Blog post.